print("Hello, World!")



LINGUAGEM DE PROGRAMAÇÃO PYTHON

AULA 09

SANTA CRUZ
CENTRO UNIVERSITÁRIO

ANTERIORMENTE

- **Devolutiva:**
  - Nota prova;
  - Gabarito e Dúvidas;

SANTA CRUZ
CENTRO UNIVERSITÁRIO

PRÓXIMOS PASSOS

- **Arquivos em Python:**
  - Abrir e fechar arquivos;
  - Conceitos Básicos: Tipos de Arquivo, Caminhos e Gerenciamento com 'with';
  - Técnicas de Leitura de Arquivos;
  - Tratamento de Exceções em Operações de Arquivo;

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Arquivos



Introdução

- **Armazenamento:** Variáveis x Arquivos.
- **Tipos de Arquivo e Caminhos (Paths).**
- **Modos de acesso:** r, w, a.
- **Gerenciador de Contexto:** O Bloco 'with'**.**
- **Métodos de Leitura.**
- **Gravação e Anexo de Dados.**
- **Tratamento de Exceções em Operações de Arquivo.**

Tipos de Arquivos

- **Texto vs. Binário:**
  - **Arquivos de Texto: Legíveis** por humanos (**.txt**, **.csv**, **.json**, .py).
  - **Arquivos Binários:** Requerem programas específicos (.png, .pdf, .mp3). Se abertos em editor de texto, parecem códigos ilegíveis.



Abrindo e fechando arquivos

- **Caminhos (Paths):**
  - **Absoluto:** Começa da raiz do sistema operacional
  - Ex: C:\Users\Batatinha\Documents\dados.txt ou /home/batatatinha/Documents/dados.txt
  - **Relativo:** Começa no diretório onde o script Python está em execução.

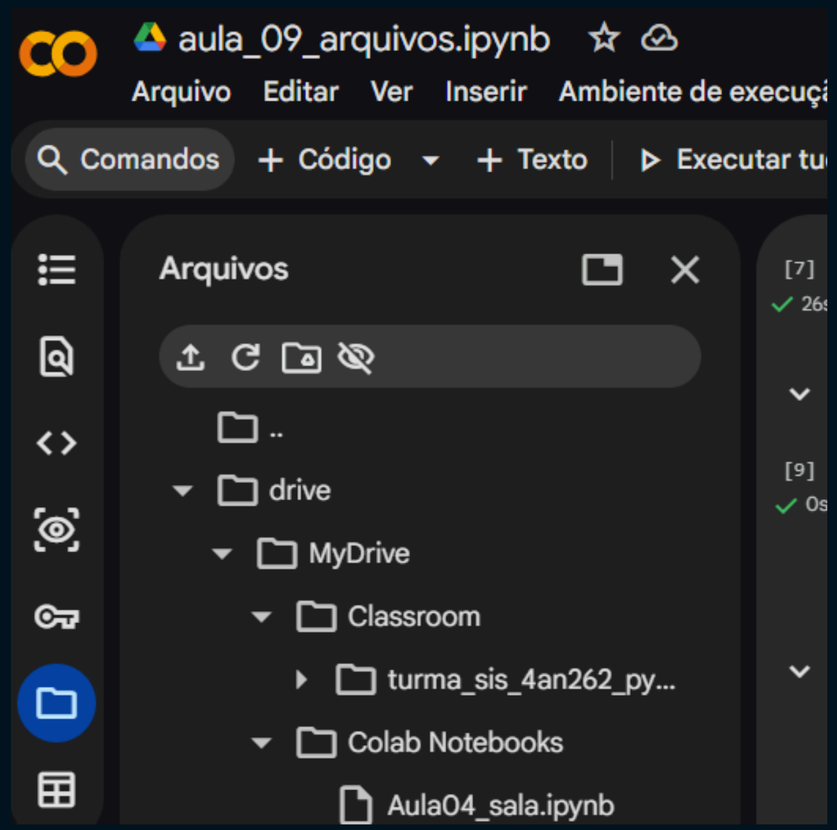
SANTA CRUZ
CENTRO UNIVERSITÁRIO

Caminhos (Paths):

```python
import os
folder = "alguma_pasta"
filename = "arquivo.txt"
path = os.path.join(folder, filename)
print("O caminho completo é:", path)
```

```
O caminho completo é: alguma_pasta/arquivo.txt
```

Caminhos (Paths):




```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
!pwd
```

```
/content
```

```
import os
print(os.getcwd())
```

```
/content
```

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Modos de Abertura de Arquivo

- **Abrir um arquivo em Python exige declarar a intenção:**
  - "r" (Read/Leitura): Modo padrão. Lê o arquivo. Lança FileNotFoundError se o arquivo não existir.
  - "w" (Write/Escrita): Cria um arquivo novo ou sobrescreve totalmente o existente (limpar tudo).
  - "a" (Append/Anexar): Adiciona dados ao final do arquivo sem apagar o conteúdo existente.
  - Modos Binários ("rb" , "wb"): Utilizados para leitura e escrita de dados binários.

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Modos de Abertura de Arquivo

	TEXTO	BINÁRIO
LEITURA + Escrita	r r+	rb rb+
ESCRITA + Leitura	w w+	wb wb+
ANEXAR (APPEND) + Leitura	a a+	ab ab+

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Modos de Abertura de Arquivo

```
arquivo = open("/content/drive/MyDrive/Colab Notebooks/arquivo.txt", "r")
conteudo = arquivo.read()
print(conteudo)
arquivo.close()

texto = "Manipulação de arquivos"

arquivo = open("/content/drive/MyDrive/Colab Notebooks/escrita.txt", "w")
arquivo.write(texto)
arquivo.close()


anexar = "\nSegunda Linha do Texto"

arquivo = open("escrita.txt", "a")
arquivo.write(anexar)
arquivo.close()
```

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Arquivos Binários

```
imagem = open("logo.png", "rb")
logo = imagem.read()
print(logo)
imagem.close()
```

```
b'\x89PNG\r\n\x1a\n\x00\x00\x00\rIHDR\x00\x00\x048\x00\x00\x03`\x08\x02\x00\x00\x00F\x08O\xcc\:
```

```
imagem_saida = open("imagem_nova.png", "wb")
imagem_saida.write(logo)
imagem_saida.close()
```

SANTA CRUZ
CENTRO UNIVERSITÁRIO

O Bloco 'with'

- **Sintaxe Tradicional:**
  - **Requer chamar file.close()** manualmente, correndo o risco de esquecer ou de o código falhar antes da linha de fechamento.

  - Com **with** (Context Manager): O Python fecha o arquivo automaticamente assim que o bloco indentado termina, evitando vazamentos de recursos.

Leitura (read), Gravação (write) e Anexo de Dados (append)

- **Modo Leitura("r"):**

```
with open("log.txt", "r") as log:
    conteudo = log.read()
    print(conteudo)
```

- **Modo Escrita ("w"):**

```
with open("log.txt", "w") as file:
     file.write("Novo Registro\n")
```

- **Atenção: Se log.txt já existia, seu conteúdo original foi apagado instantaneamente ao abrir com "w"!**

- Modo Anexo ("a"):

```
with open("log.txt", "a") as file:
 file.write("Adicionando mais um registro sem apagar nada!\n")
```

- **Ideal para construir diários, logs de auditoria e históricos contínuos.**

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Atividade Prática 01 Aula 09

Atividade Leitura e escrita

Sistema de Auditoria de Logs

Você faz parte da equipe de segurança e precisa automatizar o monitoramento de acessos a um servidor crítico. Você irá criar, atualizar e analisar o arquivo acessos.log.

Objetivo:

*Crie um script Python que execute as 4 etapas a seguir:

Etapa 1: Inicialização do Log (Modo Escrita - w)

Crie o arquivo acessos.log gravando o cabeçalho do sistema e o primeiro evento de inicialização:

Etapa 2: Registro de Atividades (Modo Append - a)

Sem apagar o conteúdo existente, adicione pelo menos 5 eventos ao final do arquivo.

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Formas de Ler um Arquivo de Texto

- **file.read() :** Lê todo o conteúdo do arquivo de uma vez só e o retorna como uma única string multinível. Ideal para arquivos pequenos.
- **file.readline() :** Lê uma única linha por vez. Mantém o ponteiro de leitura na linha seguinte.
- **file.readlines() :** Retorna uma lista de strings, onde cada elemento corresponde a uma linha do arquivo.
- **Iteração direta (for line in file:):** A forma mais eficiente em termos de memória! Processa o arquivo linha por linha sem carregar tudo na memória de uma vez

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Formas de Ler um Arquivo de Texto

```
with open("log.txt", "r", encoding="utf-8") as log:
    primeira_linha = log.readline()
    segunda_linha = log.readline()

print("1ª chamada:", primeira_linha.strip())
print("2ª chamada:", segunda_linha.strip())
```

```
1ª chamada: 2026-10-14 18:42:00 - INFO - Nova informação adic
2ª chamada: 2026-10-14 08:31:02 - INFO - Usuario 'admin' real
```

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Formas de Ler um Arquivo de Texto

```
with open("log.txt", "r", encoding="utf-8") as log:
    linhas = log.readlines()

print("Tipo:", type(linhas))
print("Quantidade de linhas:", len(linhas))
print("Última linha:", linhas[-1].strip())
print("Lista completa:", linhas)
```

Formas de Ler um Arquivo de Texto

```
with open("log.txt", "r", encoding="utf-8") as log:
    for numero, linha in enumerate(log, start=1):
        if "ERRO" in linha:
            print(f"Alerta na linha {numero}: {linha.strip()}")
```

```
Alerta na linha 5: 2026-10-14 08:40:11 - ERRO - Falha ao conectar ao ba
Alerta na linha 11: 2026-10-14 08:40:11 - ERRO - Falha ao conectar ao b
```

Atividade Prática 01 Aula 09

Atividade Leitura e escrita

Etapa 3: Leitura e Inspeção Seletiva

Abra o arquivo em modo de leitura e resolva:

1. Inspeção do cabeçalho: Usando readline(), leia e exiba apenas a primeira linha do arquivo (o título).
2. Último evento registrado: Usando readlines(), obtenha e imprima diretamente a última linha registrada no arquivo.

Etapa 4: Auditoria de Segurança

Utilizando a iteração direta (for linha in arquivo:):
Percorra o arquivo sem carregar tudo na memória de uma só vez.
Identifique e imprima no terminal apenas os eventos marcados com uma palavra-chave como: LOGIN ou ERRO.

Exiba a contagem total dessa palavra chave encontrada ao final.

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Tratamento de Exceções

Principais Erros ao Manipular Arquivos

- **Ao trabalhar com o mundo externo, coisas podem dar errado. As duas exceções mais comuns são:**
  - **FileNotFoundError :** Ocorre ao tentar abrir no modo de leitura ("r") um arquivo ou caminho que não existe.
  - **PermissionError :** Ocorre quando o sistema operacional nega acesso de leitura/escrita ao arquivo especificado.

Atividade Prática 02 Aula 09

Módulo de Resiliência de Arquivos

Etapa 1: Adicione uma camada de tratamento de erros ao script anterior utilizando blocos try / except / finally, cobrindo os seguintes cenários:

Tentativa de leitura de arquivo inexistente (FileNotFoundError):

- Tente abrir um arquivo com nome incorreto (ex.: servidor_inexistente.log) antes de ler o arquivo principal.
- Capture FileNotFoundError e mostre uma mensagem amigável ao usuário em vez de deixar o programa quebrar.

Etapa 2: Proteção de escrita e permissões (PermissionError):

- Envolva a escrita e o append em blocos protegidos para capturar restrições de escrita no diretório ou em arquivos somente-leitura.

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Atividade Prática 02 Aula 09

Módulo de Resiliência de Arquivos

Etapa 3: Tratamento de decodificação (UnicodeDecodeError):
Capture erros de decodificação caso o arquivo seja aberto com encoding incompatível.

Etapa 4: Fechamento e cleanup com finally:
Demonstrar o fechamento seguro de conexões/arquivos mesmo se ocorrer um erro inesperado.

Resumo

- **Persistência é essencial:** Salvar dados em arquivos permite manter informações mesmo após o encerramento do programa.
- **Sempre use with:** O gerenciador de contexto with garante que os arquivos sejam fechados automaticamente, prevenindo erros de vazamento de memória.
- **Cuidado com o modo "w" :** Abrir um arquivo em modo escrita apaga todo o conteúdo existente instantaneamente; use "a" para anexar novos dados.

Resumo

- **Escolha o método de leitura correto:** Use read() para arquivos pequenos e iteração direta (for line in file:) para arquivos grandes.
- **Formatos nativos:** Utilize a biblioteca padrão CSV para tabelas e JSON para estruturas de dados complexas/dicionários.
- **Trate erros:** Proteja operações de arquivo contra *FileNotFoundError* e *PermissionError* usando blocos **try-except**.

Próximos Passos

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Bibliotecas.
Arquivos Estruturados.

exit() ou dúvidas?

SANTA CRUZ
CENTRO UNIVERSITÁRIO

OBRIGADA!

“In the face of ambiguity, refuse the temptation to guess.”
The Zen of Python, by Tim Peters

SANTA CRUZ
CENTRO UNIVERSITÁRIO

SANTA CRUZ

CENTRO UNIVERSITÁRIO

Para mais informações acesso o **QRCODE** e saiba mais

(41)3052-4900

/UniSantaCruzCtba

@unisantacruz

unisantacruztem.com.br